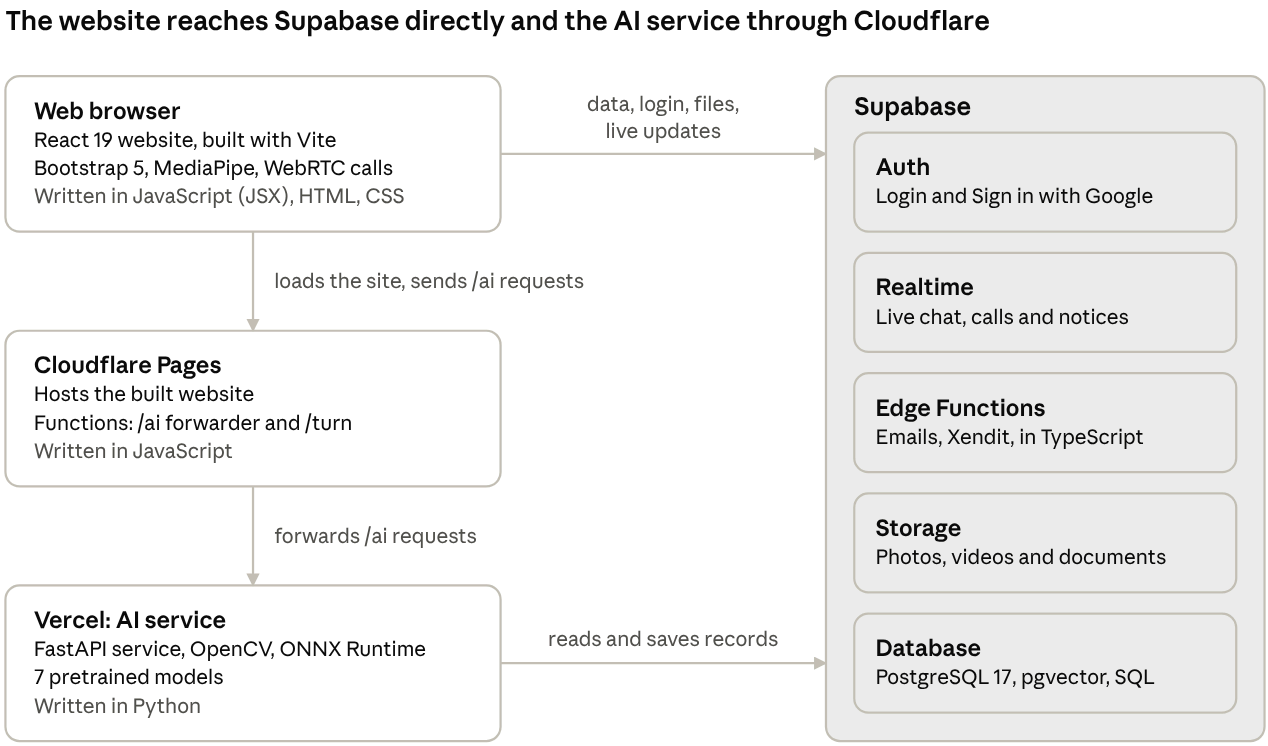
PhilFreela: Tools and Programming Languages

Oct 26, 2026 · @VG

Overview

PhilFreela is a web-based freelancing platform for Filipino freelancers and clients, built from three parts: a React website, a Supabase backend and a Python AI service.



system architecture · 3 hosts, 4 connections

The browser never calls the AI service directly. Cloudflare forwards `/ai` requests to Vercel, and both the website and the AI service read and write the same Supabase project.

Programming languages

PhilFreela is written in six languages: JavaScript for the website, Python for the AI service, SQL for the database, plus CSS, HTML and TypeScript for the Supabase server functions.

Language	Files	Where it runs	Used for
JavaScript (with JSX)	216	Browser and Cloudflare Pages	Every page and component of the website, plus two small server functions: the AI forwarder and the call relay
SQL (PostgreSQL)	127	Supabase database	Tables, row level security rules, database functions and demo data
Python	25	AI service on Vercel	Identity verification, watermarking, copy checks, moodboard matching, search, recommendations and job skill suggestions
CSS	10	Browser	The dark theme and page layouts, on top of Bootstrap
HTML	4	Browser and email	The website's page shell, a phone check page and two email templates (confirm sign-up, reset password)
TypeScript	4	Supabase Edge Functions	Three Edge Functions: offline emails and the two Xendit payment functions

File counts are the files tracked in Git on October 26, 2026.

Frontend (website)

The website is a React 19 single-page app, built with Vite and styled with Bootstrap 5 plus the project's own CSS. It lives in the `client` folder.

Tool	Version	Used for
React and React DOM	19.3.0	Building the user interface from components
React Router	7.18.3	Moving between pages without reloading
Vite	8.2.2	Development server and production build
@vitejs/plugin-react	6.1.1	React support in Vite

Tool	Version	Used for
@vitejs/plugin-basic-ssl	2.3.0	HTTPS on the development server, so phone cameras work
Bootstrap	5.3.3	Layout grid and ready-made components, loaded from the jsDelivr CDN
Bootstrap Icons	1.11.3	Icons across the site
Supabase JS client	2.116.0	Connects the website to the database, login, file storage and live updates
MediaPipe Tasks Vision	1.0.1	Tracks the face live in the browser during the face scan
qrcode.react	4.2.0	The QR code that opens identity verification on a phone

Typefaces come from Google Fonts: Plus Jakarta Sans, Syne and Audiowide.

Three features use what browsers already provide, with no extra library:

Browser feature	Used for
WebRTC	Voice and video calls between two users
Camera and microphone access	ID photos, the face scan, calls and voice messages
Canvas	Shrinking photos before upload and previewing watermarks

Backend and database (Supabase)

Supabase is the backend: one cloud project in Tokyo gives PhilFreela its PostgreSQL database, login, file storage and live updates. Apart from the AI service there is no separate backend server; the website talks to Supabase directly, and row level security decides what each user may read or change.

Supabase service	Used for
Database (PostgreSQL)	All of the system's records: users, services, job posts, chats, bookings, projects, payments, ratings, promotions, reports and admin logs
Auth	Sign-up, login, password reset emails, user sessions and Sign in with Google
Storage	Uploaded files in 16 buckets: profile and cover pictures, service and portfolio media, chat files, resumes, project deliverables, report evidence, ID photos, payment receipts, PhilFreela's GCash QR code and billboard images
Realtime	Live chat messages, the inbox, incoming calls, GCash payment records, announcements and notifications
Edge Functions	Three TypeScript functions: `send-offline-email` emails users who are offline, and the two Xendit functions take Freelancer Pro payments (see Payments)
Row level security and database functions	Access rules on every table, and server-side logic the website calls by name

The database itself and the add-ons switched on inside it:

Tool	Version	Used for
PostgreSQL	17.6	The relational database
PL/pgSQL	1.0	The language of the database functions and triggers
pgvector	0.8.2	Stores the AI models' number lists and finds the closest ones (cosine similarity, HNSW index) for search, recommendations, copy checks and moodboard matching

Tool	Version	Used for
pg_cron	1.6.4	Scheduled jobs: ending calls that were left hanging, removing expired bans, and turning down ID checks that no admin reviewed in time
pg_net	0.20.4	Lets the database call the email function
Supabase Vault	0.3.1	Keeps the email function's address and secret code encrypted
pgcrypto	1.3	Hashes the passwords of the demo accounts
Nodemailer	6.9.16	Sends the offline emails through Gmail, from inside the Edge Function

Payments (transactions)

Only Freelancer Pro goes through a payment company: it is paid online through Xendit, which runs in test mode, so no real money is charged yet. Every other payment is made in the GCash app and recorded on the site, which never touches the money.

Tool	Used for
Xendit (Payment Sessions API, test mode)	The checkout page for Freelancer Pro (₱149 for 30 days), paid with GCash, Maya or QR Ph. Cards are left out because their fee is about ₱20. PhilFreela never sees the payment details.
`pro-checkout` Edge Function (TypeScript)	Opens the Xendit page, sending only the amount and PhilFreela's own payment number, and checks the payment when the person comes back
`xendit-webhook` Edge Function (TypeScript)	Receives Xendit's "paid" or "expired" message, checks its secret token, and asks Xendit again before Pro is given
`online_checkouts` table and two database functions	Keep one row per payment page, take the price from the price list, record each payment once, and start or extend Pro

Tool	Used for
GCash app (no connection to the site)	Payments between clients and freelancers, and Featured posts. The payer uploads the receipt with its 13-digit reference; the freelancer confirms it was received, and an admin checks Featured post payments.

Still to do: set the webhook address in Xendit's dashboard and make one test-mode payment on the live site. Charging real money needs Xendit to verify the business first, which takes up to 14 days.

AI service (Python)

The AI features run in a separate Python service built with FastAPI, in the `ai-service` folder. The website reaches it at `/ai` for identity verification, watermarking, copy checks, search, recommendations, job skill suggestions and moodboard matching.

Tool	Version	Used for
Python	3.10.11	The language of the service
FastAPI	0.141.1	The web API the website calls
Uvicorn	0.53.0	The server that runs the FastAPI app
Pydantic	2.13.5	Checks the data sent in each request
OpenCV	5.0.0	Image processing: runs the face and text models, reads QR codes, measures sharpness and handles video frames
ONNX Runtime	1.23.2	Runs the HiDDeN, DINO, MiniLM and CLIP models
Tokenizers (Hugging Face)	0.23.2	Cuts text into word pieces for the text model
NumPy	2.2.6	Number and image array maths
Pillow	12.3.0	Opens, rotates and resizes photos, and draws the visible watermark
imageio-ffmpeg (FFmpeg)	0.6.0	Reads uploaded videos and writes the watermarked 720p MP4

Tool	Version	Used for
python-docx	1.2.0	Reads the text of DOCX files
pypdf	6.19.0	Reads the text of PDF files
pypdfium2 (PDFium)	5.14.0	Turns a PDF's first pages into watermarked pictures, so viewers see the real pages but never get the file
supabase-py	2.31.0	Reads and saves records in Supabase
python-dotenv	1.2.3	Loads settings from the `.env` file
python-multipart	0.0.32	Receives uploaded photos and files

Versions are the ones installed on the development laptop. PyTorch was used once, outside the service, to convert the HiDDeN model into ONNX files.

DeepFace, TensorFlow, RetinaFace and ArcFace appear in the earlier list (`PhilFreela-Tools-Used.pdf`, September 25) but are no longer used. YuNet and SFace on OpenCV replaced them.

Pretrained AI models

PhilFreela uses eight pretrained models and trains none of its own.

Model	From	Runs on	Used for
MediaPipe Face Landmarker	Google	Browser	Tracks the face live during the face scan
YuNet	OpenCV model collection	AI service	Finds faces on ID photos and face scan frames, and checks head turns
SFace	OpenCV model collection	AI service	Compares the face on the ID with the face scan
PP-OCRv3 text detection	OpenCV model collection	AI service	Finds printed text, to tell an ID card from a random photo

Model	From	Runs on	Used for
HiDDeN	Meta (Stable Signature project)	AI service	Hides an invisible 48-bit code in photos and videos, and reads it back
DINO ViT-S/16	Meta	AI service	Copy check: spots a picture that copies another freelancer's work
all-MiniLM-L6-v2	sentence-transformers	AI service	Search by meaning, recommendations, job skill suggestions and the copy check for documents
CLIP ViT-B/32 (vision half)	OpenAI	AI service	Moodboard matching: finds portfolio work that looks like a reference image

Four parts use no model, only the project's own code:

- Collaborative filtering and the ranking formula in the hybrid recommendation system (Jaccard similarity plus weighted scores)
- The invisible watermark for documents (zero-width Unicode characters)
- The sharpness check on ID photos (Laplacian variance in OpenCV)
- Reading the QR code on a PhilSys ID (OpenCV's QR reader)

Development tools

The system was written in Visual Studio Code on Windows 11, with Git and GitHub for version control and Claude Code as the AI coding assistant.

Tool	Version	Used for
Visual Studio Code	1.143.1	Code editor
Node.js	24.20.0	Runs the frontend tooling (Vite, Oxlint, Playwright)
npm	11.19.0	Installs the frontend packages
pip and venv	26.2.1	Install the Python packages in the service's own environment

Tool	Version	Used for
Git	2.55.0	Version control
Oxlint	1.82.0	Checks the JavaScript code for mistakes
Playwright	1.63.0	Opens pages in a real browser to test them
Claude Code (Anthropic)	2.1.266	AI coding assistant

Two more tools have no version number: the Supabase Dashboard and the Supabase MCP connector, both used to manage the database (running the SQL files, checking tables). The AI service also has four test scripts in `ai-service`, run with plain Python: `test_ranking.py`, `test_skills.py`, `test_document_preview.py` and `test_page_blur.py`.

Hosting and outside services

The live system runs on three hosts: Cloudflare Pages for the website, Vercel for the AI service and Supabase for the data. The live address is phil-freela.pages.dev.

Service	Used for
Cloudflare Pages	Hosts the website, plus two small functions: `/ai` passes requests on to the AI service, and `/turn` gives logged-in users the call relay servers
Vercel (free plan)	Hosts the Python AI service in Tokyo, and updates it on every push to GitHub
Supabase (cloud)	Hosts the database, login, file storage, live updates and the email and payment functions, in Tokyo
GitHub	Online code repository (`CodeniParengVenjy/Phil-Freela`)
TURN relay	Carries a voice or video call when two devices cannot connect directly; the code supports Cloudflare Realtime TURN or a fixed TURN account

Service	Used for
Google Identity Services	The Sign in with Google button
Gmail (SMTP)	Sends the emails to users who are offline
Xendit	The checkout page for Freelancer Pro, in test mode (see Payments)
jsDelivr CDN and Google Fonts	Deliver Bootstrap, Bootstrap Icons and the typefaces
Hugging Face and the OpenCV model collection	Where the AI service downloads its larger model files while it builds

On a laptop, the same system runs with `npm run dev` for the website and Uvicorn for the AI service, both still using the cloud Supabase project.